

Full Stack AI Engineer — Take-Home Assessment

Project: Personal Finance Assistant

An AI-driven, multi-user financial companion

Field	Detail
Duration	6 hours (single sitting). Plan your time — finishing everything is not the goal.
Format	Build a real, working full-stack application. Code + a short written design note.
Stack	Your choice. Use whatever you are fastest and most confident in.
What we assess	How you think: architecture, scale, context handling, model selection, and your handling of ambiguity and edge cases.

Read the whole brief before you start. The features are described as a product, not as a spec sheet. How you turn them into a system is the point.

1. Overview

We want you to build a Personal Finance Assistant: a real application that people log into, connect their financial data to, and talk to in plain language about their money. Think of products like Cleo, Copilot Money, or a smart layer on top of a banking app.

This is intentionally open-ended. We are not handing you a checklist of API endpoints to implement. We are describing a product and a set of things it should be able to do, and we want to see the system you design to make that happen. Two strong candidates may build very different things — that is fine, and expected.

There is no single correct answer. We care far more about the quality of your decisions, and your ability to explain them, than about how many features you finish.

2. What You Are Building

A working application with the following foundations:

- **Accounts and sign-in.** Users can create an account and sign in. Each user's financial data is private to them. You are free to use any authentication provider you like (for example Clerk, Auth0, Supabase, Firebase, or NextAuth) — reaching for an existing service here is encouraged, not penalised.
- **Multiple users.** The system supports many separate users, each with their own data, history, and assistant.
- **Financial data.** Users bring in their transaction history. We will provide a sample dataset (a CSV of transactions and a mock bank endpoint). Assume a real user may have anywhere from one month to several years of history.
- **A conversational assistant.** The heart of the product. Users interact with their finances by talking to an assistant in natural language, and by uploading images (for example, a photo of a receipt).

Beyond that, the architecture is up to you.

3. What the Assistant Should Be Able to Do

Below are the things a user should be able to get from the assistant. Treat these as user expectations, not as a list of functions to wire up one-to-one. Part of what we are evaluating is whether you recognise that these are not all the same kind of work.

1. **Answer questions about spending.** “How much did I spend on groceries last month?”, “What was my biggest purchase in March?”
2. **Read a receipt from a photo.** A user uploads a picture of a receipt; the assistant extracts the relevant details and records it as an expense.
3. **Surface recurring subscriptions.** Identify repeating charges the user may have forgotten about and present them clearly.

4. **Flag unusual activity.** Notice charges that look out of pattern for this user and bring them to their attention.
5. **Compare across time.** “Am I spending more than usual this month?” — which requires reasoning over a long history, not just recent data.
6. **Track a budget.** Let a user set a budget and have the assistant track it and warn when they are close to the limit.
7. **Look up unfamiliar charges.** When a user does not recognise a merchant or charge, the assistant can find out what it likely is, including looking online.
8. **Summarise finances in plain English.** A clear, human summary of where the user's money is going.
9. **Suggest where to cut back.** Concrete, numbers-backed suggestions personalised to the user.
10. **Remember user context.** If a user says “I get paid on the 1st” or “don't count rent in my food budget,” the assistant should remember and apply that later.

Note. Some of these are cheap, near-instant lookups. Others need genuine multi-step work, reading an image, or reaching outside the system. Some need only the last few weeks of data; others need years of it. We are deliberately not telling you how to handle that difference. How you do is a large part of the signal.

4. Constraints That Matter

Treat these as real product requirements, not nice-to-haves. They are what separate a demo from something that could actually ship.

- **It should feel fast.** A user asking a simple question should not wait a long time. Response time is part of the experience.
- **It should be economical to run.** Assume cost per interaction matters. A design that is expensive on every single request will not scale to a real user base.
- **It should hold up as data grows.** It must still work well for a user with years of transactions — not just the small sample we provide. Assume the data could be 10× to 100× larger than what you test with.
- **It should handle many users.** Consider what happens when there are many users active at once, not one person clicking through a demo.

These constraints pull against each other on purpose. Resolving that tension is the interesting part.

5. Expect the Unexpected

Real systems meet messy inputs and changing requirements. Your design should anticipate this. Among the situations we expect a robust solution to cope with:

- A receipt photo that is blurry, rotated, partly cut off, or in another language.
- A transaction dataset with duplicates, missing fields, odd formatting, or junk rows.
- An ambiguous or under-specified question from the user.
- A question that the assistant genuinely cannot answer from the data available.
- Two sources of information that contradict each other.
- A request that would be slow or expensive if handled naively.

We may also introduce a change or a new input during the session to see how your design adapts. Build something you would not be afraid to change.

6. Deliverables

1. **Working code in a public GitHub repository.** Push your work to a public GitHub repo and share the link — there is no file to upload. Include clear setup instructions so we can run it. Commit as you go, so we can see how the project progressed. A narrow slice that genuinely works beats a broad set of half-finished features.
2. **Documentation inside the repository.** Your write-up must live inside the GitHub repo itself (for example as the README, or as a Markdown file in the repo) — do not submit a separate document outside the repo. It should explain your approach and progress.

Documentation Requirement

Along with the implementation, you are required to include a short write-up inside your GitHub repository (as the README or a Markdown file in the repo — not a separate document) explaining your approach and progress. This should include:

- What features/tasks you covered and how much was completed.
- Key architectural and technical decisions you made, and why.
- Assumptions, trade-offs, and limitations in your approach.
- What was intentionally skipped, stubbed, or simplified due to time constraints.
- Any challenges faced and how you handled them.

We care about your thinking process as much as the final output, so clarity in explaining your decisions matters.

Scoping is part of the test. Six hours is not enough to build all of this well. We want to see what you choose to build, what you choose to fake or stub, and what you choose to skip — and whether you can defend those calls.

7. What We Are Evaluating

We are not counting features. We are reading how you think. Specifically:

Area	What strong looks like
System & scalability design	Thinks in terms of throughput, cost, and latency — not “it ran once on my machine.” Design degrades gracefully under load and data growth.
Handling large context	Recognises that a long history cannot be processed all at once, and has a deliberate strategy for working with data far larger than the context available.
Routing & model selection	Matches the right level of effort and the right model to each kind of task, under the cost and speed constraints — rather than applying the heaviest approach to everything.
Multi-step / agentic reasoning	Breaks complex requests into steps, gathers what it needs, decides when it has enough, and recovers when a step fails — without being told to.
Edge-case & failure handling	Anticipates messy inputs, ambiguity, contradictions, and dead ends, and handles them gracefully instead of crashing or guessing silently.
Pragmatism (build vs. buy)	Spends effort on the genuinely hard parts and uses sensible existing tools for the commodity parts (e.g. authentication).
Adaptability	Reacts well to a changed requirement or a new input: refactors cleanly rather than freezing or hacking.
Communication	The design note is clear, honest about trade-offs and limits, and explains the why behind decisions.

A note on commodity work:

Reaching for an off-the-shelf service for things like authentication is a perfectly good decision — it is what we would expect. We want your six hours spent on the hard, interesting parts of this problem, and we are watching to see that you can tell the two apart.

8. Ground Rules

- Use any language, framework, library, or paid service you like.
- You are free to use external and managed services — authentication providers, databases, model APIs, OCR, search, and so on. Using sensible off-the-shelf services for commodity work is a good decision, not a shortcut we penalise.
- You may research freely and read documentation — we expect you to.
- AI coding assistants are allowed; you own and must be able to explain every decision in your submission.
- If you make an assumption, write it down in your design note rather than waiting to ask.

- Commit your work to the public repo as you go, so we can see how it progressed.

Good luck. Build something you would be proud to defend.